FIAP ON

BANCO DE DADOS ORACLE

COMANDOS DML

11

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Visão física da tabela DEPARTAMENTO 13
Figura 2 – Visualização das colunas da tabela Departamento no SQL Developer ...14
Figura 3 – Resultado da consulta à tabela DEPARTAMENTO 15
Figura 4 – Visão física da tabela PROJETO 15
Figura 5 – Resultado da consulta à tabela PROJETO 16
Figura 6 – Visão física da tabela FUNCIONARIO 17
Figura 7 – Resultado da consulta à tabela FUNCIONARIO 18
Figura 8 – Visão física da tabela PROJETO 19
Figura 9 – Resultado da consulta à tabela PROJETO 20
Figura 10 – Resultado da consulta à tabela PROJETO 21
Figura 11 – Resultado da consulta à tabela COPY_PROJETO_FIN 24
Figura 12 – Resultado da consulta à tabela FUNCIONARIO 25
Figura 13 – Resultado da consulta à tabela FUNCIONARIO 26
Figura 14 – Resultado da consulta à tabela PROJETO 28
Figura 15 – Resultado da consulta à tabela PROJETO 29
Figura 16 – Resultado da consulta à tabela PROJETO 30
Figura 17 – Resultado da consulta à tabela PROJETO 31
Figura 18 – Sugestão de conteúdo para a tabela DEPARTAMENTO.....40
Figura 19 – Sugestão de conteúdo para a tabela FUNCIONARIO.....41
Figura 20 – Sugestão de conteúdo para a tabela DEPENDENTE 41
Figura 21 – Sugestão de conteúdo para a tabela PROJETO.....41
Figura 22 – Sugestão de conteúdo para a tabela IMPLANTACAO 42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo comando SQL – Comando INSERT.....6
Código-fonte 2 – Exemplo comando SQL – Comando UPDATE.....7
Código-fonte 3 – Exemplo comando SQL – Comando DELETE.....8
Código-fonte 4 – SCRIPT DE CRIAÇÃO DO BANCO DE DADOS.....12
Código-fonte 5 – Exemplo comando SQL – Comando INSERT completo.....13
Código-fonte 6 – Exemplo comando SQL – Comando INSERT.....14
Código-fonte 7 – Exemplo comando SQL – Comando SELECT.....14
Código-fonte 8 – Exemplo comando SQL – Comando INSERT com manipulação de data/hora.....16
Código-fonte 9 – Exemplo comando SQL – Comando SELECT.....16
Código-fonte 10 – Exemplo comando SQL – Comando INSERT com manipulação de valores decimais.....18
Código-fonte 11 – Exemplo comando SQL – Comando SELECT.....18
Código-fonte 12 – Exemplo comando SQL – Comando INSERT com manipulação de campo opcional – método implícito.....19
Código-fonte 13 – Exemplo comando SQL – Comando SELECT.....20
Código-fonte 14 – Exemplo comando SQL – Comando INSERT com manipulação de campo opcional – método explicito.....20
Código-fonte 15 – Exemplo comando SQL – Comando SELECT.....21
Código-fonte 16 – SCRIPT DE CRIAÇÃO DA TABELA EXEMPLO.....22
Código-fonte 17 – Exemplo comando SQL – Comando INSERT com subquery.....23
Código-fonte 18 – Exemplo comando SQL – Comando INSERT com subquery.....23
Código-fonte 19 – Exemplo comando SQL – Comando SELECT.....24
Código-fonte 20 – Exemplo comando SQL – Comando UPDATE envolvendo várias colunas.....25
Código-fonte 21 – Exemplo comando SQL – Comando SELECT.....25
Código-fonte 22 – Exemplo comando SQL – Comando UPDATE envolvendo cálculo.....26
Código-fonte 23 – Exemplo comando SQL – Comando SELECT.....26
Código-fonte 24 – Exemplo comando SQL – Comando UPDATE a partir de uma subconsulta.....27
Código-fonte 25 – Exemplo comando SQL – Comando SELECT.....27
Código-fonte 26 – Exemplo comando SQL – Comando DELETE.....29
Código-fonte 27 – Exemplo comando SQL – Comando SELECT.....29
Código-fonte 28 – Exemplo comando SQL – Comando DELETE.....30
Código-fonte 29 – Exemplo comando SQL – Comando SELECT.....30
Código-fonte 30 – Exemplo comando SQL – Comando ROLLBACK.....30
Código-fonte 31 – Exemplo comando SQL – Comando SELECT.....31
Código-fonte 32 – Exemplo comando SQL – Erro de violação de Primary Key.....32
Código-fonte 33 – Exemplo comando SQL – Erro de violação de Unique Constraint.....33
Código-fonte 34 – Exemplo comando SQL – Erro de violação de Foreign Key.....34
Código-fonte 35 – Exemplo comando SQL – Erro de violação de Check Constraint 35
Código-fonte 36 – Exemplo comando SQL – Erro relativo à precisão de um campo 36
Código-fonte 37 – Exemplo comando SQL – Erro de tipo de dados inválido.....37
Código-fonte 38 – Exemplo comando SQL – Erro de valores insuficientes para inserção.....38

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Código-fonte 39 – Exemplo comando SQL – Erro de valores demais para inserção 39
Código-fonte 40 – Exemplo comando SQL – Populando as tabelas do banco de dados SIP 48
Código-fonte 41 – Exemplo comando SQL – Treinando inserções 49
Código-fonte 42 – Exemplo comando SQL – Treinando inserções 50
Código-fonte 43 – Exemplo comando SQL – Treinando inserções 51
Código-fonte 44 – Exemplo comando SQL – Treinando atualizações 51
Código-fonte 45 – Exemplo comando SQL – Treinando atualizações 51
Código-fonte 46 – Exemplo comando SQL – Treinando exclusões 52
Código-fonte 47 – Exemplo comando SQL – Treinando ROLLBACK 52
Código-fonte 48 – Exemplo comando SQL – Treinando exclusões 53

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 COMANDOS DML....6
1.1 Inclusão ou inserção de dados....6
1.2 Alteração ou atualização de dados....7
1.3 Exclusão ou deleção de dados....8
1.4 Exemplos de aplicação dos comandos DML....9
1.5 Inserção de dados em tabelas....12
1.6 Inserção de dados com manipulação de conteúdo data/hora....15
1.7 Inserção de dados com manipulação de conteúdo numérico decimal....17
1.8 Inserção de dados com manipulação de campos opcionais....18
1.9 Inserção de dados a partir de uma subconsulta....21
1.10 Atualização de dados em tabelas....24
1.11 Atualização de dados envolvendo várias colunas....25
1.12 Atualização de dados envolvendo cálculos....26
1.13 Atualização de dados a partir de uma subconsulta....27
1.14 Exclusão de linhas em tabelas....28
1.15 Confirmação e descarte de transações....29
1.16 Erros comuns ao executar INSERT, UPDATE e DELETE....31
1.16.1 Erros de violação de PRIMARY KEY....31
1.16.2 Erros de violação de UNIQUE CONSTRAINT....32
1.16.3 Erros de violação de FOREIGN KEY....33
1.16.4 Erros de violação de CHECK CONSTRAINT....34
1.16.5 Erros relativo à precisão de um campo....35
1.16.6 Erro de tipo de dados inválido....36
1.16.7 Erro referente a valores insuficientes para inserção....37
1.16.8 Erro referente a valores demais para inserção....38
1.17 Hands On – Manipulação de Dados....39
1.17.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS....39
1.17.1.1 Populando as tabelas com comandos DML....39
1.17.1.2 Manipulando os dados com comandos DML....48
REFERÊNCIAS....54
GLOSSÁRIO....55

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 COMANDOS DML

Na linguagem SQL, temos instruções que nos permitem manipular os dados de um banco de dados. Essas instruções pertencem à divisão DML (Data Manipulation Language – Linguagem de Manipulação de Dados) da linguagem SQL.

São operações de manutenção de um banco de dados:

- Inclusão ou inserção de dados em uma tabela.
- Alteração ou atualização de dados em uma tabela.
- Exclusão ou deleção de dados em uma tabela.

Após a implementação da estrutura de um banco de dados através de um SGBD, como o Oracle Database, o passo seguinte é popular este banco de dados. Populamos um banco de dados utilizando as instruções DML, que são: INSERT, UPDATE e DELETE.

1.1 Inclusão ou inserção de dados

Para inserirmos dados em uma tabela, utilizamos o comando INSERT, que é um comando dentro da divisão DML, da linguagem SQL.

Sintaxe básica do comando INSERT:

```
INSERT INTO  <nome-tabela>
    [(<nome-coluna>, [<nome-coluna>],...)]
VALUES
    (<conteúdo>, [<conteúdo>], ...);
```

Código-fonte 1 – Exemplo comando SQL – Comando INSERT
Fonte: PRICE (2009)

Onde:

<nome-tabela>: Tabela onde será efetuada a inserção do registro ou linha.

<nome-coluna>: Colunas que terão os valores atribuídos.

<conteúdo>: Relação de valores a serem incluidos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Importante:
Possíveis situações de ERROS durante a execução dos comandos DML.

Comando INSERT:
Violação de chave primária (**Primary Key**).
Violação de **UNIQUE** constraint.
Valor informado excede o tamanho de coluna.
Tipo de dado incompatível com a coluna (caracteres em coluna).
Violação de **CHECK** constraint.
Violação de chave estrangeira (**Foreign Key**).
Violação de **NOT NULL**.

1.2 Alteração ou atualização de dados

Para atualizarmos os dados em uma tabela, utilizamos o comando UPDATE, que é um comando dentro da divisão DML, da linguagem SQL.

Sintaxe básica do comando UPDATE:

```
UPDATE <nome-tabela>
SET <nome-coluna> = <novo conteúdo para o campo>
  [,<nome-coluna> = <novo conteúdo para o campo>]
[WHERE <condição>]
```

Código-fonte 2 – Exemplo comando SQL – Comando UPDATE
Fonte: PRICE, J. Oracle Database 11g SQL 1 (2009)

Onde:

<nome-tabela>: Tabela onde será efetuada a atualização dos dados do registro ou linha.

<nome-coluna>: Colunas que terão os valores atualizados.

<conteúdo>: Novo conteúdo atribuído a coluna.

<condição>: Critério que indicará quais linhas serão afetadas, durante a atualização.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Importante:
Possíveis situações de ERROS durante a execução dos comandos DML.

Comando UPDATE:
Violação de chave primária (**Primary Key**).
Violação de **UNIQUE** constraint.
Valor informado excede o tamanho de coluna.
Tipo de dado incompatível com a coluna (caracteres em coluna).
Violação de **CHECK** constraint.
Violação de chave estrangeira (**Foreign Key**).
Violação de **NOT NULL**.

1.3 Exclusão ou deleção de dados

Para excluirmos os dados em uma tabela, utilizamos o comando DELETE, que é um comando dentro da divisão DML, da linguagem SQL.

Sintaxe básica do comando DELETE:

```
DELETE FROM  <nome-tabela>
     [WHERE <condição>]
```

Código-fonte 3 – Exemplo comando SQL – Comando DELETE
Fonte: PRICE (2009)

Onde:

<nome-tabela>: Tabela onde será efetuada a exclusão do registro ou linha.

<condição>: Critério que indicará quais linhas serão afetadas, durante a exclusão.

Importante:
Possíveis situações de ERROS durante a execução dos comandos DML.

Comando DELETE:
Violação de chave estrangeira (**Foreign Key**).

Durante a manipulação dos dados, devemos considerar:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Quando um erro ocorrer, o registro (linha) não será inserido/alterado/excluído da base de dados.

O comando INSERT/UPDATE/DELETE é efetivado somente após um COMMIT.

É possível desfazer a operação de INSERT/UPDATE/DELETE através do comando ROLLBACK. Mas atenção, após o COMMIT das transações, não será possível desfazer operações, nem mesmo utilizando o ROLLBACK.

Em alguns SGBDR´s, as ferramentas disponibilizadas podem realizar o COMMIT automático (por exemplo SQL Server). Em programação, pode ser necessário o uso de comandos COMMIT e/ou ROLLBACK.

Para realizar as operações o usuário deve ser o proprietário (owner) da tabela, ou seja, precisa ter criado a tabela em seu esquema de banco de dados, ou possuir privilégios para executar os comandos.

Um esquema, ou “SCHEMA”, é uma coleção de objetos dentro de um banco de dados. Sua função é agrupar objetos que se referem a áreas de interesse comum dentro de um modelo de dados.

1.4 Exemplos de aplicação dos comandos DML

Para exemplificar as aplicações dos comandos DML, iremos construir (implementar) uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DDL_PROJETO_SIP.SQL”, com a extensão SQL.

Apresentamos abaixo o código na linguagem SQL, que permite implementar a estrutura de banco de dados do PROJETO SIP.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- SCRIPT PARA CRIAÇÃO DO BANCO DE DADOS

-- DO ESTUDO DE CASO SIP

-- COMANDOS PARA EXCLUIR AS TABELAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS;
DROP TABLE t_sip_dependente CASCADE CONSTRAINTS;
DROP TABLE t_sip_funcionario CASCADE CONSTRAINTS;
DROP TABLE t_sip_implantacao CASCADE CONSTRAINTS;
DROP TABLE t_sip_projeto CASCADE CONSTRAINTS;

-- COMANDOS PARA CRIAR AS TABELAS

-- E ADICIONAR CONSTRAINTS: PRIMARY KEY, CHECK E UNIQUE

CREATE TABLE t_sip_departamento (
    cd_depto NUMBER(3)       NOT NULL,
    nm_depto VARCHAR2(30)    NOT NULL,
    sg_depto CHAR(3)         NOT NULL
);

COMMENT ON COLUMN t_sip_departamento.cd_depto IS
    'Um departamento é identificado por um código numérico de
três dígitos.';

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT pk_sip_departamento
   PRIMARY KEY ( cd_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_nome
        UNIQUE ( nm_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_sigla
        UNIQUE ( sg_depto );

CREATE TABLE t_sip_dependente (
    nr_matricula  NUMBER(5)      NOT NULL,
    cd_dependente NUMBER(2)      NOT NULL,
    nm_dependente VARCHAR2(50)   NOT NULL,
    dt_nascimento DATE           NOT NULL
);

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT pk_sip_dependente
   PRIMARY KEY ( nr_matricula, cd_dependente );
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Comandos DML

```
CREATE TABLE t_sip_funcionario (
    nr_matricula      NUMBER(5)      NOT NULL,
    cd_depto          NUMBER(3)      NOT NULL,
    nm_funcionario    VARCHAR2(50)   NOT NULL,
    dt_nascimento     DATE           NOT NULL,
    dt_admissao       DATE           NOT NULL,
    ds_endereco       VARCHAR2(100)  NOT NULL,
    vl_salario_mensal NUMBER(7, 2) NOT NULL
);

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT ck_sip_func_salario
         CHECK ( vl_salario_mensal >= 1212 );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT pk_sip_funcionario
   PRIMARY KEY ( nr_matricula );

CREATE TABLE t_sip_implantacao (
    cd_projeto     NUMBER(5) NOT NULL,
    cd_implantacao NUMBER(3) NOT NULL,
    nr_matricula   NUMBER(5) NOT NULL,
    dt_entrada     DATE      NOT NULL,
    dt_saida       DATE          NULL
);

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT ck_sip_implant_data
         CHECK ( dt_saida > dt_entrada );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT pk_sip_implantacao
   PRIMARY KEY ( cd_projeto, cd_implantacao );

CREATE TABLE t_sip_projeto (
    cd_projeto NUMBER(5)    NOT NULL,
    nm_projeto VARCHAR2(40) NOT NULL,
    dt_inicio  DATE         NOT NULL,
    dt_termino DATE
);

-- ADICIONAR CONSTRAINT FOREIGN KEY

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT ck_sip_projeto_data
         CHECK ( dt_termino > dt_inicio );

   ALTER TABLE t_sip_projeto
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
ADD CONSTRAINT pk_sip_projeto
   PRIMARY KEY ( cd_projeto );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT un_sip_projeto_nome
        UNIQUE ( nm_projeto );

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT fk_sip_dependente_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT fk_sip_func_depto
   FOREIGN KEY ( cd_depto )
    REFERENCES t_sip_departamento ( cd_depto );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_projeto
   FOREIGN KEY ( cd_projeto )
    REFERENCES t_sip_projeto ( cd_projeto );
```

Código-fonte 4 – SCRIPT DE CRIAÇÃO DO BANCO DE DADOS
Fonte: Elaborado pela autora (2023)

1.5 Inserção de dados em tabelas

Realizamos a inserção de dados em uma tabela através do comando INSERT.

Para inserir dados em uma tabela, o usuário precisa ser proprietário da tabela ou possuir privilégios para inserção de dados.

Utilizaremos o estudo de caso SIP (Sistema de Controle de Implantação de Projetos) para exemplificar as operações realizadas por meio das instruções SQL.

Para executar as instruções SQL para inserção de dados, utilizaremos a ferramenta SQL Developer da Oracle.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É importante visualizarmos a tabela na sua visão física, isso facilita na criação do comando, pois é possível visualizar, além da sua estrutura física, os nomes de todos os elementos que compõem essa tabela.

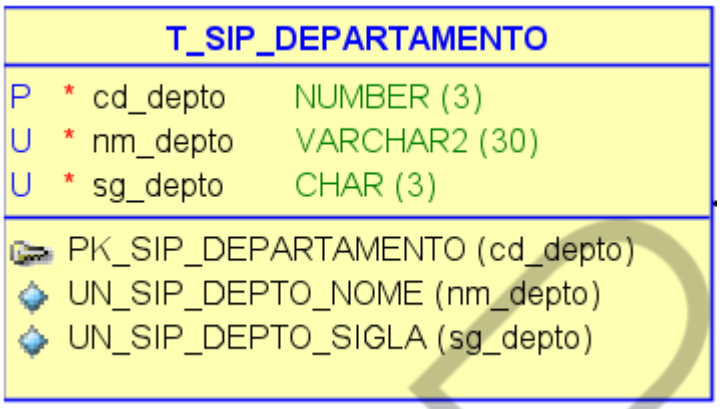

Figura 1 – Visão física da tabela DEPARTAMENTO
Fonte: Elaborado pela autora (2023)

Podemos realizar a inserção informando ou não as colunas. Quando não informamos as colunas no comando INSERT, os valores das colunas devem ser inseridos no comando, conforme a ordem de criação da tabela. Quando informamos as colunas no comando INSERT, os valores das colunas devem ser inseridos na ordem que as colunas foram declaradas no comando.

É importante destacar que valores contendo cadeia de caracteres devem ser informados entre apóstrofos.

Exemplificaremos o comando INSERT, para inserir registros ou linhas na tabela “DEPARTAMENTO”.

No exemplo abaixo, declaramos o nome de todas as colunas no comando INSERT.

Os valores (conteúdo) para cada coluna deve ser informado na mesma ordem da declaração, separados por vírgula.

```
-- INSERINDO DADOS NA TABELA DEPARTAMENTO INFORMANDO
-- COLUNAS E RESPECTIVO CONTEÚDO
INSERT INTO T_SIP_DEPARTAMENTO (CD_DEPTO, NM_DEPTO, SG_DEPTO)
           VALUES (2, 'MARKETING','MKT');
```

Código-fonte 5 – Exemplo comando SQL – Comando INSERT completo
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No exemplo abaixo apresentamos o comando INSERT, omitindo a declaração do nome das colunas no momento da inserção. Para que o comando seja executado com sucesso, é necessário que seja respeitado a ordem física das colunas ao inserir os respectivos valores (conteúdo).

Para auxiliar a elaboração do comando, recomenda-se que liste as colunas da tabela através da ferramenta SQL Developer, conforme imagem apresentada.

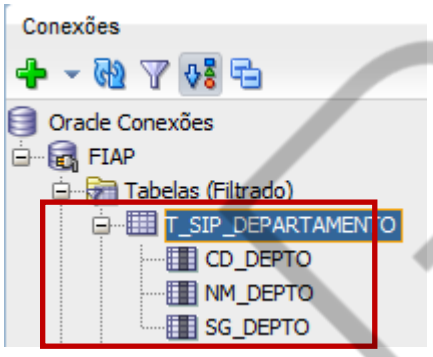

Figura 2 – Visualização das colunas da tabela Departamento no SQL Developer
Fonte: Elaborado pela autora (2023)

```
-- INSERINDO DADOS NA TABELA DEPARTAMENTO INFORMANDO
-- COLUNAS E RESPECTIVO CONTEÚDO
INSERT INTO T_SIP_DEPARTAMENTO (CD_DEPTO, NM_DEPTO, SG_DEPTO)
          VALUES (2, 'MARKETING','MKT');
```

Código-fonte 6 – Exemplo comando SQL – Comando INSERT
Fonte: Elaborado pela autora (2023)

Utilize o comando SELECT para realizar a consulta simples na tabela “DEPARTAMENTO”.

O comando SELECT será abordado mais adiante, mas para visualizar a inserção realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA DEPARTAMENTO
SELECT * FROM T_SIP_DEPARTAMENTO;
```

Código-fonte 7 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nesse exemplo, estamos consultando todas as colunas (campos) e linhas da tabela “DEPARTAMENTO”.

Onde:

Asterisco (*): equivale a todas as colunas da tabela “DEPARTAMENTO”.

Resultado da execução da consulta, através do comando “SELECT”:

	CD_DEPTO	NM_DEPTO	SG_DEPTO
1	1	FINANCEIRO	FIN
2	2	MARKETING	MKT

Figura 3 – Resultado da consulta à tabela DEPARTAMENTO
Fonte: Elaborado pela autora (2023)

EFETIVANDO AS TRANSAÇÕES DE INSERÇÃO NA TABELA
Após inserir informações, sempre aplique o comando COMMIT, para que a inserção física seja realizada, caso contrário, ao fechar a ferramenta, as informações inseridas, serão perdidas.

1.6 Inserção de dados com manipulação de conteúdo data/hora

Exemplificaremos o comando INSERT com a manipulação de conteúdo data/hora para inserir registros ou linhas na tabela “PROJETO”.

Iremos visualizar a tabela na sua visão física para utilizar corretamente os nomes de cada elemento e visualizar detalhes que sejam relevantes na estrutura.

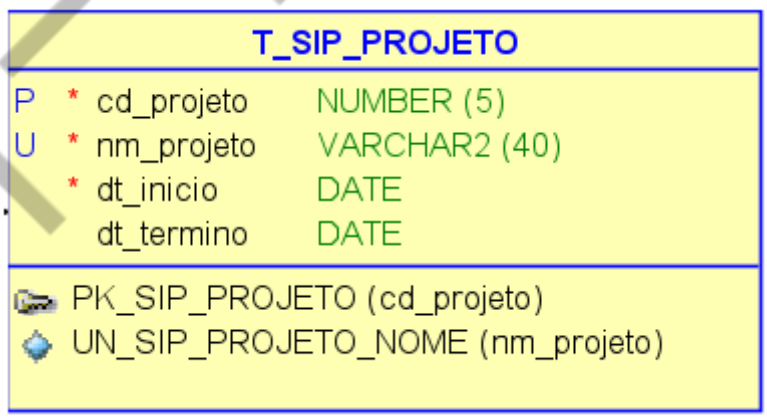

Figura 4 – Visão física da tabela PROJETO
Fonte: Elaborado pela autora (2023)

Quando manipulamos data/hora, devemos utilizar a função TO_DATE() do Oracle para informar o formato da data manipulada. Isso é importantíssimo para que

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

não tenhamos problemas relativos a configuração de data/hora no servidor Oracle Database.

Utilizamos a função TO_DATE(), informamos a data e o formato que esta data está sendo enviada ao SGBDR.

A função `TO_DATE(data,formato)` é uma função do Oracle (PL/SQL) que converte o parâmetro informado em uma data no formato especificado.

No exemplo realizamos a inserção de projetos na tabela e informamos as datas de início e término, utilizando a função `TO_DATE()`.

```
-- INSERÇÃO COM MANIPULAÇÃO DE DATA/HORA
INSERT INTO T_SIP_PROJETO
     VALUES (1, 'PROJETO ABC',
             TO_DATE('01/02/2020','DD/MM/YYYY'),
             TO_DATE('15/11/2022','DD/MM/YYYY'));
```

Código-fonte 8 – Exemplo comando SQL – Comando INSERT com manipulação de data/hora
Fonte: Elaborado pela autora (2023)

Para visualizar a inserção realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA PROJETO
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 9 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, por meio do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	1	PROJETO ABC	01/02/20	15/11/22

Figura 5 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.7 Inserção de dados com manipulação de conteúdo numérico decimal

Exemplificaremos o comando INSERT com a manipulação de conteúdo data/hora, para inserir registros ou linhas na tabela “FUNCIONARIO”.

Iremos visualizar a tabela na sua visão física, para utilizar corretamente os nomes de cada elemento e visualizar detalhes que sejam relevantes na estrutura.

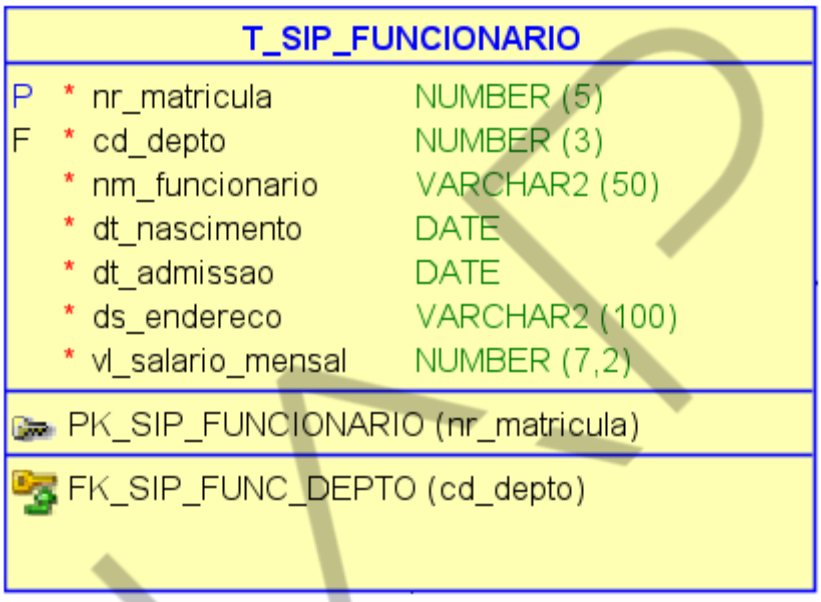

Figura 6 – Visão física da tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

Para a inserção de valores decimais, normalmente, informamos o valor a ser armazenado, utilizando o ponto decimal.

Não armazenamos formatação no banco de dados. A máscara para formatação poderá ser aplicada no momento da consulta aos dados no banco ou na apresentação dos dados ao usuário (camada de apresentação dos dados na aplicação).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- INSERÇÃO COM MANIPULAÇÃO DE NÚMEROS DECIMAIS
INSERT INTO T_SIP_FUNCIONARIO
     VALUES (12345,1,'JOÃO DA SILVA XAVIER',
             TO_DATE('18/03/1985','DD/MM/YYYY'),
             TO_DATE('21/07/2012','DD/MM/YYYY'),
             'AVENIDA PAULISTA, 1900 - 10o. ANDAR',
             1234.56); -- UTILIZAMOS O PONTO DECIMAL
```

Código-fonte 10 – Exemplo comando SQL – Comando INSERT com manipulação de valores decimais
Fonte: Elaborado pela autora (2023)

Para visualizar a inserção realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA FUNCIONARIO
SELECT * FROM T_SIP_FUNCIONARIO;
```

Código-fonte 11 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, por meio do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	12345	1	JOÃO DA SILVA XAVIER	18/03/85	21/07/12	AVENIDA PAULISTA, 1900 - 10o. ANDAR	1234,56

Figura 7 – Resultado da consulta à tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

1.8 Inserção de dados com manipulação de campos opcionais

Exemplificaremos o comando INSERT com a manipulação de campos opcionais, para inserir registros ou linhas na tabela “PROJETO”.

Iremos visualizar a tabela na sua visão física, para utilizar corretamente os nomes de cada elemento e visualizar detalhes que sejam relevantes na estrutura.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

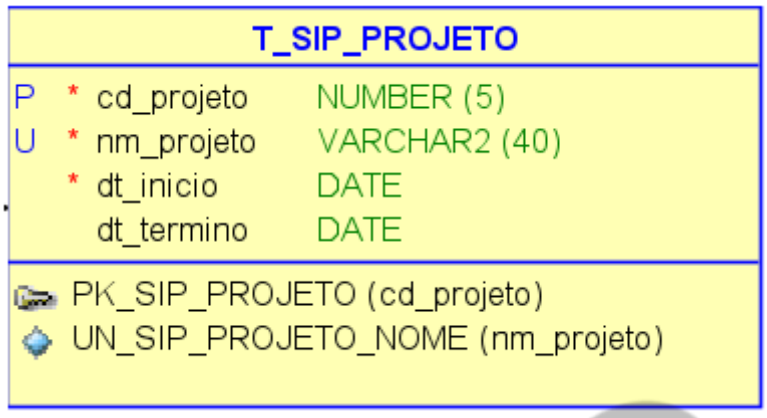

Figura 8 – Visão física da tabela PROJETO
Fonte: Elaborado pela autora (2023)

Quando manipulamos campos opcionais, podemos ou não preencher valores para esse campo. Quando um campo opcional não possui um valor, normalmente preenchemos este campo com “NULL”.

Podemos manipular campos opcionais e informar a ausência de valor de duas formas: através do método implícito ou método explicito.

No método implícito, omitimos o campo opcional no momento da inserção dos dados. Quando utilizamos esse método, uma vez que o campo é omitido, o conteúdo do campo será preenchido com “NULL”, que representa a ausência de valor.

No exemplo, omitimos o campo “DT_TERMINO” (campo opcional) na declaração das colunas para inserção. Uma vez que a coluna não foi declarada, não deve ser informado o valor da data término na lista de valores da cláusula “VALUES”, durante a inserção.

```
-- INSERÇÃO DE VALORES NULOS PARA CAMPOS OPCIONAIS
-- MÉTODO IMPLÍCITO - OMITA A COLUNA DA LISTA DURANTE A INSERÇÃO
INSERT INTO T_SIP_PROJETO (CD_PROJETO, NM_PROJETO, DT_INICIO)
     VALUES (2, 'PROJETO DEF',
             TO_DATE('19/03/2020','DD/MM/YYYY'));
```

Código-fonte 12 – Exemplo comando SQL – Comando INSERT com manipulação de campo opcional – método implícito
Fonte: Elaborado pela autora (2023)

Para visualizar a inserção realizada, podemos executar o comando SELECT abaixo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA PROJETO
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 13 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	2	PROJETO DEF	19/03/20	(null)
2	1	PROJETO ABC	01/02/20	15/11/22

Figura 9 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

No método explicito declaramos a palavra-chave “NULL”, na cláusula “VALUES” no momento da inserção.

No exemplo declaramos a palavra-chave “NULL” para o valor do campo “DT_TERMINO”.

```
-- INSERÇÃO DE VALORES NULOS PARA CAMPOS OPCIONAIS
-- MÉTODO EXPLICITO - ESPECIFIQUE A PALAVRA-CHAVE NULL,
-- NA CLÁUSULA VALUES
INSERT INTO T_SIP_PROJETO
    VALUES (3, 'PROJETO GHI',
            TO_DATE('15/08/2021','DD/MM/YYYY'),
            NULL );
```

Código-fonte 14 – Exemplo comando SQL – Comando INSERT com manipulação de campo opcional – método explicito
Fonte: Elaborado pela autora (2023)

Para visualizar a inserção realizada, podemos executar o comando SELECT abaixo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA PROJETO
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 15 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	2	PROJETO DEF	19/03/20	(null)
2	3	PROJETO GHI	15/08/21	(null)
3	1	PROJETO ABC	01/02/20	15/11/22

Figura 10 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

1.9 Inserção de dados a partir de uma subconsulta

Para exemplificar a inserção de dados a partir de uma subconsulta, aplicando os comandos DML, iremos construir (implementar) uma estrutura de banco de dados.

Apresentamos abaixo o código na linguagem SQL, que permite implementar a estrutura de banco de dados utilizada para este exemplo.

```
-- CRIAR UMA TABELA EXEMPLO
-- A TABELA ARMAZENARÁ APENAS PROJETOS FINALIZADOS
CREATE TABLE T_SIP_COPY_PROJETO_FIN
(
    CD_PROJETO          NUMBER(5)       NOT NULL ,
    NM_PROJETO          VARCHAR2(40)    NOT NULL ,
    DT_INICIO           DATE            NOT NULL ,
    DT_TERMINO          DATE            NOT NULL
);

-- CHAVE PRIMARIA DA TABELA
   ALTER TABLE T_SIP_COPY_PROJETO_FIN
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
ADD CONSTRAINT PK_SIP_COPY_PROJETO_FIN
   PRIMARY KEY (CD_PROJETO);
```

Código-fonte 16 – SCRIPT DE CRIAÇÃO DA TABELA EXEMPLO
Fonte: Elaborado pela autora (2023)

Exemplo de comando INSERT, utilizando uma subconsulta (subquery). Com esta instrução, todas as linhas da tabela “PROJETO” serão copiadas para a tabela “T_SIP_COPY_PROJETO_FIN”.

Observe que utilizamos um comando SELECT (consulta), por meio do qual recuperamos todas as linhas da tabela “PROJETO”, esta instrução de consulta é o que chamamos de subquery.

As linhas recuperadas através desta consulta serão inseridas na tabela “T_SIP_COPY_PROJETO_FIN”.

No exemplo, iremos copiar os dados referentes aos projetos finalizados apenas. Portanto, não iremos copiar todas as linhas da tabela “PROJETO”.

Apresentaremos esse exemplo por meio de duas instruções SQL, comandos DML equivalentes.

Atenção, escolha uma das formas apresentadas para realizar o teste de execução do comando.

Na primeira instrução SQL, declaramos todas as colunas no comando “INSERT” e no comando “SELECT”. É importante ressaltar que as colunas das tabelas sejam equivalentes e estejam na mesma ordem para a instrução ser executada com sucesso.

```
-- REALIZANDO A CÓPIA DOS DADOS DE UMA TABELA PARA OUTRA
-- UTILIZANDO O COMANDO INSERT E O COMANDO SELECT
-- SUBCONSULTA OU SUBQUERY)
INSERT INTO T_SIP_COPY_PROJETO_FIN
       (CD_PROJETO, NM_PROJETO, DT_INICIO, DT_TERMINO)
       (SELECT CD_PROJETO, NM_PROJETO,
               DT_INICIO, DT_TERMINO
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            FROM T_SIP_PROJETO
           WHERE DT_TERMINO IS NOT NULL);
```

Código-fonte 17 – Exemplo comando SQL – Comando INSERT com subquery
Fonte: Elaborado pela autora (2023)

Na segunda instrução SQL, omitimos as colunas no comando “INSERT” e no comando “SELECT”, podemos escrever a instrução desta maneira, devido as colunas das duas tabelas estarem na mesma ordem. É importante ressaltar, que as colunas das tabelas sejam equivalentes e estejam na mesma ordem, para a instrução ser executada com sucesso.

```
-- REALIZANDO A CÓPIA DOS DADOS DE UMA TABELA PARA OUTRA
-- UTILIZANDO O COMANDO INSERT E O COMANDO SELECT
-- SUBCONSULTA OU SUBQUERY)
INSERT INTO T_SIP_COPY_PROJETO_FIN
            (SELECT * FROM T_SIP_PROJETO
              WHERE DT_TERMINO IS NOT NULL);
```

Código-fonte 18 – Exemplo comando SQL – Comando INSERT com subquery
Fonte: Elaborado pela autora (2023)

A cláusula “WHERE” no comando “SELECT” permite criar filtros durante a consulta dos dados.

O operador especial “IS NOT NULL” permite validar se o conteúdo da coluna (campo) não é nulo.

Como a ideia é recuperar apenas os projetos finalizados, é necessário validar se os projetos possuem a data término informada. Por esse motivo, a validação realizada por meio da cláusula “WHERE” avalia se a coluna (campo) data término não é nulo ("IS NOT NULL”), ou seja, se possui uma data informada.

Para visualizar a inserção realizada, podemos executar o comando SELECT abaixo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA
-- CONTENDO OS PROJETOS FINALIZADOS
SELECT * FROM T_SIP_COPY_PROJETO_FIN;
```

Código-fonte 19 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	1	PROJETO ABC	01/02/20	15/11/22

Figura 11 – Resultado da consulta à tabela COPY_PROJETO_FIN
Fonte: Elaborado pela autora (2023)

1.10 Atualização de dados em tabelas

Realizamos a atualização de dados em uma tabela por meio do comando UPDATE.

Para atualizar dados em uma tabela, o usuário precisa ser proprietário da tabela ou possuir privilégios para inserção de dados.

Utilizaremos o estudo de caso SIP (Sistema de Controle de Implantação de Projetos) para exemplificar as operações realizadas por meio das instruções SQL.

Para executar as instruções SQL para a alteração de dados, utilizaremos a ferramenta SQL Developer da Oracle.

A cláusula “WHERE” no comando “UPDATE”, garante que serão modificadas apenas os registros (linhas) que atendem a condição.

Em todo comando “UPDATE”, recomendamos muita atenção e, ao menos que seja necessário, atualizar todos os registros da tabela em todas as outras situações sempre informe a condição que será considerada para a aplicação da alteração, caso contrário, todas as linhas da tabela serão afetadas pela alteração.

Vale ressaltar que as mesmas observações feitas durante a inserção para conteúdo numérico decimal, cadeia de caracteres, data e campos opcionais, são válidas para as instruções que envolvem atualização dos dados.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.11 Atualização de dados envolvendo várias colunas

Exemplificaremos o comando UPDATE envolvendo duas ou mais colunas (campos), para realizar a modificação/atualização dos dados armazenados.

Para atualizar vários dados de uma só vez, informe os pares nome da coluna (campo) e valor, separados por vírgula.

Com a instrução apresentada, são alterados: nome e data de nascimento de um funcionário. O funcionário que tem o número de matrícula igual a “12345”.

No exemplo realizamos a alteração dos dados na tabela e informamos o conteúdo do nome e da data, lembre-se de utilizar a função `TO_DATE()` para alteração da data.

```
-- ATUALIZAR DUAS OU MAIS COLUNAS EM UM COMANDO DE ATUALIZACAO
-- ALTERAR O NOME E A DATA DE NASCIMENTO DE UM
-- FUNCIONARIO ESPECÍFICO
   UPDATE T_SIP_FUNCIONARIO
      SET NM_FUNCIONARIO = 'JOÃO DA SILVA XAVIER DE CAMARGO',
          DT_NASCIMENTO = TO_DATE ('18/05/1985','DD/MM/YYYY')
    WHERE NR_MATRICULA = 12345;
```

Código-fonte 20 – Exemplo comando SQL – Comando UPDATE envolvendo várias colunas
Fonte: Elaborado pela autora (2023)

Para visualizar a atualização realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA FUNCIONÁRIO
SELECT * FROM T_SIP_FUNCIONARIO WHERE NR_MATRICULA = 12345;
```

Código-fonte 21 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, através do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	12345	1	JOÃO DA SILVA XAVIER DE CAMARGO	18/05/85	21/07/12	AVENIDA PAULISTA, 1900 - 10o. ANDAR	1234,56

Figura 12 – Resultado da consulta à tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.12 Atualização de dados envolvendo cálculos

Exemplificaremos o comando UPDATE envolvendo cálculo, atualizando uma coluna (campo) com base nela mesma.

Com a instrução apresentada, será alterado o salário de todos os funcionários, onde os salários serão reajustados em 5%. O cálculo é baseado o valor existente na coluna (campo) “VL_SALARIO_MENSAL”, acrescido de 5%.

Observe que não há cláusula WHERE no comando UPDATE, isto faz com que todos os salários armazenados sejam alterados.

Sugerimos que compare os salários armazenados antes e depois da execução da instrução SQL, para atualização.

```
-- ATUALIZAR O SALÁRIO DOS FUNCIONARIOS
-- TODOS OS FUNCIONARIOS IRÃO RECEBER UM AUMENTO DE 5%
UPDATE T_SIP_FUNCIONARIO
   SET VL_SALARIO_MENSAL = VL_SALARIO_MENSAL * 1.05;
-- NÃO TEMOS O WHERE (CONDICAO), POIS O AUMENTO
-- SERÁ APLICADO A TODOS FUNCIONARIOS
```

Código-fonte 22 – Exemplo comando SQL – Comando UPDATE envolvendo cálculo
Fonte: Elaborado pela autora (2023)

Para visualizar a atualização realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO TODAS AS LINHAS DA TABELA FUNCIONÁRIO
SELECT * FROM T_SIP_FUNCIONARIO ;
```

Código-fonte 23 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, através do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	12345	1	JOÃO DA SILVA XAVIER DE CAMARGO	18/05/85	21/07/12	AVENIDA PAULISTA, 1900 - 10o. ANDAR	1296,29

Figura 13 – Resultado da consulta à tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.13 Atualização de dados a partir de uma subconsulta

Exemplificaremos o comando UPDATE, atualizando os dados a partir de uma subconsulta (subquery).

Com a instrução apresentada, será alterada a data término dos projetos ainda não finalizados (projetos que mão possuem valor na data término) na tabela “PROJETO”. Para exemplificar a subconsulta, iremos utilizar a data término do projeto finalizado (projeto de código igual a 1), que está armazenado na tabela “T_SIP_COPY_PROJETO_FIN”.

Observe que utilizamos um comando SELECT (consulta), onde recuperamos a data término do projeto finalizado na tabela “T_SIP_COPY_PROJETO_FIN”. Esta instrução de consulta é o que chamamos de subquery.

```
-- ATUALIZACAO UTILIZANDO SUBCONSULTA
UPDATE T_SIP_PROJETO
   SET DT_TERMINO =
       (SELECT DT_TERMINO
          FROM T_SIP_COPY_PROJETO_FIN
         WHERE CD_PROJETO = 1)
 WHERE DT_TERMINO IS NULL;
```

Código-fonte 24 – Exemplo comando SQL – Comando UPDATE a partir de uma subconsulta
Fonte: Elaborado pela autora (2023)

Para visualizar a atualização realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO A ALTERAÇÃO REALIZADA
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 25 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução da consulta, através do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	2	PROJETO DEF	19/03/20	15/11/22
2	3	PROJETO GHI	15/08/21	15/11/22
3	1	PROJETO ABC	01/02/20	15/11/22

Figura 14 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

1.14 Exclusão de linhas em tabelas

Realizamos a exclusão (deleção/remoção/eliminação) de linhas de dados em uma tabela, através do comando DELETE.

Para apagar dados em uma tabela, o usuário precisa ser proprietário da tabela ou possuir privilégios para inserção de dados.

Utilizaremos o estudo de caso SIP (Sistema de Controle de Implantação de Projetos) para exemplificar as operações realizadas através das instruções SQL.

Para executar as instruções SQL para a exclusão de dados, utilizaremos a ferramenta SQL Developer da Oracle.

A cláusula “WHERE” no comando “DELETE”, garante que serão eliminados apenas os registros (linhas) que atendem a condição.

Em todo comando “DELETE”, recomendamos muita atenção, e ao menos que seja necessário, eliminar todos os registros da tabela em todas as outras situações sempre informe a condição que será considerada para a aplicação da remoção dos dados, caso contrário, todas as linhas da tabela serão afetadas pela exclusão.

Atenção: a execução de um comando “DELETE” sem “WHERE” todos os registros (linhas) da tabela serão apagados.

Vale ressaltar que as mesmas observações feitas durante a inserção para conteúdo numérico decimal, cadeia de caracteres, data e campos opcionais, são válidas para as instruções que envolvem a exclusão dos dados.

Exemplificaremos o comando DELETE, excluindo todos os projetos iniciados até “31/12/2022”.

Sugerimos que compare os projetos armazenados antes e depois da execução da instrução SQL, para exclusão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXCLUSÃO DOS PROJETOS INICIADOS ATÉ 31/12/2020
DELETE FROM T_SIP_PROJETO
      WHERE DT_INICIO <= TO_DATE ('31/12/2020','DD/MM/YYYY');
```

Código-fonte 26 – Exemplo comando SQL – Comando DELETE
Fonte: Elaborado pela autora (2023)

Para visualizar a exclusão realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO A EXCLUSÃO REALIZADA
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 27 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	3	PROJETO GHI	15/08/21	15/11/22

Figura 15 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

1.15 Confirmação e descarte de transações

Utilizaremos o estudo de caso SIP (Sistema de Controle de Implantação de Projetos) para exemplificar as operações realizadas por meio das instruções SQL.

Para executar as instruções SQL para a confirmação e descarte dos dados, utilizaremos a ferramenta SQL Developer da Oracle.

As operações INSERT, UPDATE e DELETE são responsáveis por dar início às transações. As instruções COMMIT e ROLLBACK controlam as transações.

Portanto todas as instruções executadas até agora, estão apenas na memória.

Para confirmamos o armazenamento permanente das transações realizadas, é necessário executar a instrução “COMMIT”, que efetivará as manipulações de dados realizadas nas tabelas fisicamente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Enquanto a instrução “ROLLBACK” descarta as transações que ainda não foram efetivadas.

Exemplificaremos o descarte das transações não efetivadas, através da deleção da tabela “PROJETO”. Observe que após a execução da deleção, a tabela “PROJETO” está vazia.

```
-- DELEÇÃO DA TABELA PROJETO INTEIRA,
-- SEM EFETIVAÇÃO DA TRANSAÇÃO
DELETE FROM T_SIP_PROJETO;
```

Código-fonte 28 – Exemplo comando SQL – Comando DELETE
Fonte: Elaborado pela autora (2023)

Para visualizar a exclusão realizada, podemos executar o comando SELECT abaixo:

```
-- CONSULTANDO A EXCLUSÃO REALIZADA
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 29 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

CD_PROJ...	NM_PROJ...	DT_INICIO	DT_TERM...

Figura 16 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

Execute a instrução “ROLLBACK” e consulte novamente a tabela “PROJETO”, observe que a deleção realizada foi descartada e os dados anteriores a deleção, retornaram para a tabela.

```
-- EXEMPLO COM ROLLBACK
ROLLBACK;
```

Código-fonte 30 – Exemplo comando SQL – Comando ROLLBACK
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para visualizar o descarte da transação, podemos executar o comando SELECT abaixo:

```
-- VISUALIZAÇÃO DA DELEÇÃO DESCARTADA
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 31 – Exemplo comando SQL – Comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	2	PROJETO DEF	19/03/20	(null)
2	3	PROJETO GHI	15/08/21	(null)
3	1	PROJETO ABC	01/02/20	15/11/22

Figura 17 – Resultado da consulta à tabela PROJETO
Fonte: Elaborado pela autora (2023)

1.16 Erros comuns ao executar INSERT, UPDATE e DELETE

É comum nos depararmos com erros durante a execução dos comandos da linguagem SQL. Alguns erros são bastante comuns e para facilitar a identificação e correção desses erros, apresentamos algumas situações em que eles ocorrem.

Utilizaremos o estudo de caso SIP (Sistema de Controle de Implantação de Projetos) para exemplificar as operações realizadas por meio das instruções SQL.

Para executar as instruções SQL demonstrando os erros comuns, utilizaremos a ferramenta SQL Developer da Oracle.

1.16.1 Erros de violação de PRIMARY KEY

O erro de violação de chave primária ocorre na tentativa de inserção de um novo registro (linha) ou atualização de um registro já existente, utilizando uma chave primária já existente (cadastrada) na tabela.

A mensagem de erro apresenta o texto “Restrição exclusiva <nome-da-constraint> violada”.

Ter nomeado adequadamente os objetos de banco de dados, facilitará a identificação da restrição em que o erro ocorre.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplificando o erro de violação de chave primária. No exemplo, estamos tentando inserir um novo departamento com um código (código igual a 1) que já existe cadastrado na tabela “Departamento”.

```
-- ERRO DE VIOLAÇÃO DE CHAVE PRIMARIA
INSERT INTO T_SIP_DEPARTAMENTO VALUES (1, 'TESTE','TST');

/*
Erro a partir da linha : 145 no comando -
INSERT INTO T_SIP_DEPARTAMENTO VALUES (1, 'TESTE','TST')
Relatório de erros -
ORA-00001: restrição exclusiva (PF0102.PK_T_SIP_DEPARTAMENTO)
violada
*/
```

Código-fonte 32 – Exemplo comando SQL – Erro de violação de Primary Key
Fonte: Elaborado pela autora (2023)

1.16.2 Erros de violação de UNIQUE CONSTRAINT

O erro de violação de “unique constraint” ocorre na tentativa de inserção de um novo registro (linha) ou atualização de um registro já existente, utilizando um conteúdo que não pode ser repetido na coluna (campo) declarada como “unique constraint”.

A mensagem de erro apresenta o texto “Restrição exclusiva <nome-da-constraint> violada”.

Ter nomeado adequadamente os objetos de banco de dados, facilitará a identificação da restrição em que o erro ocorre.

Exemplificando o erro de violação de “unique constraint”. No exemplo, estamos tentando inserir um departamento, utilizando um nome de departamento já cadastrado na tabela “DEPARTAMENTO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO DE VIOLAÇÃO DE UNIQUE CONSTRAINT
INSERT INTO T_SIP_DEPARTAMENTO VALUES (3, 'MARKETING', 'MKT');
/*
Erro a partir da linha : 154 no comando -
INSERT INTO T_SIP_DEPARTAMENTO VALUES (3, 'MARKETING', 'MKT')
Relatório de erros -
ORA-00001: restrição exclusiva (PF0102.UN_SIP_DEPTO_NOME)
violada
*/
```

Código-fonte 33 – Exemplo comando SQL – Erro de violação de Unique Constraint
Fonte: Elaborado pela autora (2023)

1.16.3 Erros de violação de FOREIGN KEY

O erro de violação de chave estrangeira ocorre na tentativa de inserção de um novo registro (linha) ou atualização de um registro já existente, utilizando um valor para a chave estrangeira, que não está definido como um valor para a chave primária.

A mensagem de erro apresenta o texto “Restrição de integridade <nome-da-constraint> violada - chave mãe não localizada”.

Ter nomeado adequadamente os objetos de banco de dados, facilitará a identificação da restrição em que o erro ocorre.

Exemplificando o erro de violação de chave estrangeira. No exemplo, estamos tentando inserir um funcionário, utilizando um código de departamento como chave estrangeira, que não está cadastrado na tabela “DEPARTAMENTO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO DE VIOLAÇÃO DE CHAVE ESTRANGEIRA
INSERT INTO T_SIP_FUNCIONARIO
     VALUES  (12346, 99 ,'JOSÉ CLEMENTE',
              TO_DATE('21/03/1987','DD/MM/YYYY'),
              TO_DATE('04/05/2020','DD/MM/YYYY'),
              'AVENIDA PAULISTA, 1900 - 10o. ANDAR',
              3541.55);
/*
Erro a partir da linha : 163 no comando -
INSERT ...
Relatório de erros -
ORA-02291: restrição de integridade (PF0102.FK_SIP_FUNC_DEPTO)
violada - chave mãe não localizada
*/
```

Código-fonte 34 – Exemplo comando SQL – Erro de violação de Foreign Key
Fonte: Elaborado pela autora (2023)

1.16.4 Erros de violação de CHECK CONSTRAINT

O erro de violação de “check constraint” ocorre na tentativa de inserção de um novo registro (linha) ou atualização de um registro já existente, utilizando um valor, que viola a regra de validação estabelecida na “check constraint”.

A mensagem de erro apresenta o texto “Restrição de verificação <nome-da-constraint> violada”.

Ter nomeado adequadamente os objetos de banco de dados, facilitará a identificação da restrição em que o erro ocorre.

Exemplificando o erro de violação de “check constraint”. No exemplo, estamos tentando inserir um funcionário, informando um salário inferior ao valor definido na regra de validação “check constraint” (no exemplo o valor do salário mensal de um funcionário deve ser maior ou igual a R$ 1212,00).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO DE VIOLAÇÃO DE CHECK CONSTRAINT
INSERT INTO T_SIP_FUNCIONARIO
     VALUES (12347, 1 ,'JOSÉ CLEMENTE',
              TO_DATE('21/03/1987','DD/MM/YYYY'),
              TO_DATE('04/05/2020','DD/MM/YYYY'),
              'AVENIDA PAULISTA, 1900 - 10o. ANDAR',
              599.99);
/*
Erro a partir da linha : 180 no comando -
INSERT ...
Relatório de erros -
ORA-02290:          restrição          de          verificação
(PF0102.CK_SIP_FUNC_SALARIO) violada
*/
```

Código-fonte 35 – Exemplo comando SQL – Erro de violação de Check Constraint
Fonte: Elaborado pela autora (2023)

1.16.5 Erros relativo à precisão de um campo

O erro relativo à precisão de um campo, ocorre na tentativa de inserção de um novo registro (linha) ou atualização de um registro já existente, utilizando um valor com a precisão que excede o tamanho máximo especificado para a coluna (campo).

A mensagem de erro apresenta o texto “valor maior que a precisão especificada usado para a coluna”.

Exemplificando o erro relativo à precisão de um campo. No exemplo, estamos tentando inserir um valor para o campo código do departamento, superior ao tamanho máximo informado que é de três dígitos numéricos inteiro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO REFERENTE A PRECISÃO DO CAMPO
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (12345, 'DEPTO TESTE', 'TST');
/*
Erro a partir da linha : 194 no comando -
INSERT INTO T_SIP_DEPARTAMENTO VALUES (12345, 'DEPTO TESTE',
'TST')
Relatório de erros -
ORA-01438: valor maior que a precisão especificada usado para
esta coluna
*/
```

Código-fonte 36 – Exemplo comando SQL – Erro relativo à precisão de um campo
Fonte: Elaborado pela autora (2023)

1.16.6 Erro de tipo de dados inválido

O erro de tipo de dados inválido ocorre na tentativa de inserção de um novo registro (linha) ou atualização de um registro já existente, utilizando um valor cujo conteúdo não é compatível com a definição do tipo de dados referente a coluna (campo).

A mensagem de erro apresenta o texto “número inválido”.

Exemplificando o erro de tipo de dados inválido. No exemplo, estamos tentando inserir um valor para o campo código do departamento, com o conteúdo em caracteres, mas o campo foi definido como um numérico inteiro de três dígitos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO DE TIPO DE DADOS INVÁLIDO
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES ('XXX', 'DEPTO TESTE', 'TST');
/*
Erro a partir da linha : 204 no comando -
INSERT INTO T_SIP_DEPARTAMENTO VALUES ('XXX', 'DEPTO TESTE',
'TST')
Relatório de erros -
ORA-01722: número inválido
*/
```

Código-fonte 37 – Exemplo comando SQL – Erro de tipo de dados inválido
Fonte: Elaborado pela autora (2023)

1.16.7 Erro referente a valores insuficientes para inserção

O erro referente a valores insuficientes para inserção ocorre na tentativa de inserção de um novo registro (linha), informando menos valores (conteúdo dos campos) do que o esperado, conforme a estrutura da tabela.

A mensagem de erro apresenta o texto “não há valores suficientes – not enough values”.

Exemplificando o erro referente a valores insuficientes para inserção. No exemplo, estamos tentando inserir apenas dois valores (para código e nome do departamento), conforme a estrutura da tabela, são esperados três valores, uma vez que temos três campos definidos na sua estrutura.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO DE VALORES INSUFICIENTES PARA A INSERÇÃO
INSERT INTO T_SIP_DEPARTAMENTO VALUES (222, 'DEPTO TESTE');
/*
Erro na Linha de Comandos : 213 Coluna : 13
Relatório de erros -
Erro de SQL: ORA-00947: não há valores suficientes
00947. 00000 -  "not enough values"
*Cause:
*Action:
*/
```

Código-fonte 38 – Exemplo comando SQL – Erro de valores insuficientes para inserção
Fonte: Elaborado pela autora (2023)

1.16.8 Erro referente a valores demais para inserção

O erro referente a valores demais para inserção, ocorre na tentativa de inserção de um novo registro (linha), informando mais valores (conteúdo dos campos) do que o esperado, conforme a estrutura da tabela.

A mensagem de erro apresenta o texto “valores demais – too many values”.

Exemplificando o erro referente a valores demais para inserção. No exemplo, estamos tentando inserir apenas quatro valores, conforme a estrutura da tabela, são esperados três valores, uma vez que temos três campos definidos na sua estrutura.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- ERRO DE VALORES DEMAIS PARA A INSERÇÃO
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (5, 'DEPTO TESTE', 'TST', 'XXX');
/*
Erro na Linha de Comandos : 225 Coluna : 13
Relatório de erros -
Erro de SQL: ORA-00913: valores demais
00913. 00000 -  "too many values"
*Cause:
*Action:
*/
```

Código-fonte 39 – Exemplo comando SQL – Erro de valores demais para inserção
Fonte: Elaborado pela autora (2023)

1.17 Hands On – Manipulação de Dados

1.17.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS

1.17.1.1 Populando as tabelas com comandos DML

No item 1.4, utilizamos o “SCRIPT DDL”, para criar a estrutura do estudo de caso SIP, para exemplificarmos instruções para manipulação de dados.

Iremos popular toda a estrutura do banco de dados SIP, utilizando os comandos da divisão DML, desenvolvidos como exemplo nas seções anteriores.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DML_PROJETO_SIP.SQL”, com a extensão SQL.

Sugerimos que o arquivo “SCRIPT”, seja elaborado na seguinte sequência, considerando as boas práticas:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Posicionar no início do arquivo SCRIPT, os comandos “DELETE”, para eliminar os registros (linhas) da tabela. Os comandos “DELETE”, podem ser mantidos em bloco de comentários (/*...*/), para serem utilizados quando necessário.
- Para cada tabela do banco de dados do projeto SIP, criar o comando INSERT, para popular as tabelas. Sugerimos utilizar os exemplos apresentados nesta seção.
- As tabelas que possuírem colunas (campos) do tipo de dado “DATE”, deverão ter seus conteúdos inseridos, utilizando a função “`TO_DATE(<data>,<formato>)`”.
- Executar o SCRIPT e garantir que todos os comandos foram executados com sucesso.
- Após a execução dos comandos para popular as tabelas do banco de dados do projeto SIP, efetive as transações pendentes, criando e executando o comando “COMMIT”.

Exemplos com sugestões para popular cada tabela do banco de dados do projeto SIP.

Popular a tabela “DEPARTAMENTO”, inserindo cinco departamentos, conforme exemplo.

	CD_DEPTO	NM_DEPTO	SG_DEPTO
1	1	FINANCEIRO	FIN
2	2	TECNOLOGIA DA INFORMAÇÃO	TIN
3	3	CONTAS A PAGAR	CPG
4	4	FATURAMENTO	FAT
5	5	RECURSOS HUMANOS	RHU

Figura 18 – Sugestão de conteúdo para a tabela DEPARTAMENTO
Fonte: Elaborado pela autora (2023)

Popular a tabela “FUNCIONARIO”, inserindo três funcionários para cada departamento criado, conforme exemplo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
2	12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
3	12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
4	12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
5	12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
6	12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
7	12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
8	12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
9	12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
10	12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
11	12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12	12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
13	12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
14	12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
15	12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 19 – Sugestão de conteúdo para a tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

Popular a tabela “DEPENDENTE”, escolhendo três funcionários quaisquer e inserindo para cada funcionário escolhido, três dependentes, conforme exemplo.

	NR_MATRICULA	CD_DEPENDENTE	NM_DEPENDENTE	DT_NASCIMENTO
1	12345	1	JOANINHA	12/10/10
2	12345	2	JULINHA	15/10/12
3	12345	3	TONINHO	22/10/14
4	12356	1	JUNINHO	02/10/11
5	12356	2	ZEZINHO	15/10/13
6	12356	3	MARCELINHO	17/10/15
7	12359	1	MARIAZINHA	11/06/14
8	12359	2	LUIZINHA	21/06/16
9	12359	3	CARMINHA	18/06/18

Figura 20 – Sugestão de conteúdo para a tabela DEPENDENTE
Fonte: Elaborado pela autora (2023)

Popular a tabela “PROJETO”, inserindo cinco projetos, onde três deles não devem ter data término, ou seja, não devem ter sido finalizados, conforme exemplo.

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	10001	PROJETO 1	11/10/16	(null)
2	10002	PROJETO 2	25/10/13	15/04/15
3	10003	PROJETO 3	28/04/12	(null)
4	10004	PROJETO 4	25/03/17	(null)
5	10005	PROJETO 5	28/09/14	15/09/18

Figura 21 – Sugestão de conteúdo para a tabela PROJETO
Fonte: Elaborado pela autora (2023)

Popular a tabela “IMPLANTAÇÃO”, escolhendo três projetos quaisquer e para cada projeto escolhido, inserir três funcionários trabalhando em diferentes implantações, conforme exemplo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Atenção: Para o período de entrada e saída de cada funcionário que trabalha em um projeto, considerar sua data de admissão (um funcionário não pode trabalhar em um projeto antes de ser admitido) e considerar a data de início e término (se existir) dos projetos, para alocação dos funcionários (um funcionário não pode trabalhar em um projeto que ainda não iniciado ou iniciar em um projeto depois do seu término).

	CD_PROJETO	CD_IMPLANTACAO	NR_MATRICULA	DT_ENTRADA	DT_SAIDA
1	10001	1	12345	11/10/16	15/09/17
2	10001	2	12346	11/10/16	(null)
3	10001	3	12348	11/10/16	25/06/18
4	10002	1	12345	25/10/13	15/04/14
5	10002	2	12352	25/10/13	25/07/14
6	10002	3	12355	25/10/13	15/04/15
7	10003	1	12354	28/04/13	(null)
8	10003	2	12355	28/04/13	(null)
9	10003	3	12345	28/04/13	(null)
10	10004	1	12357	25/03/17	(null)
11	10004	2	12358	25/03/17	(null)
12	10004	3	12354	25/03/17	15/08/18
13	10005	1	12345	28/09/14	15/09/18
14	10005	2	12350	28/09/14	15/09/18
15	10005	3	12354	28/09/14	15/09/18

Figura 22 – Sugestão de conteúdo para a tabela IMPLANTACAO
Fonte: Elaborado pela autora (2023)

Apresentamos abaixo o código na linguagem SQL, que permite popular as tabelas do banco de dados do PROJETO SIP.

```
-- SCRIPT PARA POPULAR O BD IMPLANTACAO DE PROJETOS

-- ELIMINA O CONTEUDO DAS TABELAS
/*
DELETE FROM T_SIP_IMPLANTACAO;
DELETE FROM T_SIP_PROJETO;
DELETE FROM T_SIP_DEPENDENTE;
DELETE FROM T_SIP_FUNCIONARIO;
DELETE FROM T_SIP_DEPARTAMENTO;
*/

-- POPULAR A TABELA DEPTO COM 5 DEPARTAMENTOS
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (1, 'FINANCEIRO', 'FIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (2, 'TECNOLOGIA DA INFORMAÇÃO','TIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (3, 'CONTAS A PAGAR','CPG');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (4, 'FATURAMENTO','FAT');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (5, 'RECURSOS HUMANOS', 'RHU');

-- POPULAR A TABELA FUNCIONARIO COM 3 FUNCIONARIOS PARA CADA
DEPTO
-- DEPTO = 1
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12345,         1,         'JOAO         DA         SILVA',
TO_DATE('10/05/1985','DD/MM/YYYY'),
     TO_DATE('15/09/2012','DD/MM/YYYY','RUA X, 49', 5684.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12346,         1,         'MANUEL        DA        SILVA',
TO_DATE('05/10/1998','DD/MM/YYYY'),
     TO_DATE('10/11/2015','DD/MM/YYYY','RUA X, 31', 3542.11);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12347,         1,         'JANDIRA        DA        SILVA',
TO_DATE('10/12/2000','DD/MM/YYYY'),
     TO_DATE('15/09/2018','DD/MM/YYYY','RUA X, 25', 1875.96);

-- DEPTO = 2
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12348,         2,         'KATIA        REGINA        SOUZA',
TO_DATE('15/01/1995','DD/MM/YYYY'),
     TO_DATE('03/10/2015','DD/MM/YYYY','RUA Y, 49', 3894.63);
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12349,      2,      'MARIA      DAS      DORES      SOUZA',
TO_DATE('18/08/1983','DD/MM/YYYY'),
    TO_DATE('23/10/2017','DD/MM/YYYY'),'RUA Y, 35', 1542.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12350,        2,        'ALFREDO        DE        SOUZA',
TO_DATE('04/05/1999','DD/MM/YYYY'),
    TO_DATE('05/06/2014','DD/MM/YYYY'),'RUA Y, 27', 5874.52);

-- DEPTO = 3
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12351,        3,        'GISELE        DE        JESUS',
TO_DATE('15/04/1999','DD/MM/YYYY'),
    TO_DATE('20/03/2017','DD/MM/YYYY'),'RUA Z, 49', 1360.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12352,        3,        'RAFAEL        DE        JESUS',
TO_DATE('10/08/1998','DD/MM/YYYY'),
    TO_DATE('10/08/2012','DD/MM/YYYY'),'RUA Z, 55', 2563.44);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12353,        3,        'ROSANA        DE        JESUS',
TO_DATE('14/03/1987','DD/MM/YYYY'),
    TO_DATE('15/08/2019','DD/MM/YYYY'),'RUA Z, 79', 4879.55);

-- DEPTO = 4
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12354,       4,       'JOSEFINA       DE       ALMEIDA',
TO_DATE('16/10/1997','DD/MM/YYYY'),
    TO_DATE('25/03/2013','DD/MM/YYYY'),'RUA Y, 33', 4561.88);

INSERT INTO T_SIP_FUNCIONARIO VALUES
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    (12355,        4,        'LUCIANA        DE        ALMEIDA',
TO_DATE('10/02/1984','DD/MM/YYYY'),
    TO_DATE('28/09/2011','DD/MM/YYYY'),'RUA Y, 44', 2345.52);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12356,        4,        'THIAGO        DE        ALMEIDA',
TO_DATE('10/03/1998','DD/MM/YYYY'),
    TO_DATE('24/10/2018','DD/MM/YYYY'),'RUA Y, 55', 1254.22);

-- DEPTO = 5
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12357,        5,        'LARISSSA        DE        CAMARGO',
TO_DATE('14/02/1997','DD/MM/YYYY'),
    TO_DATE('04/08/2015','DD/MM/YYYY'),'RUA V, 22', 1245.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12358,        5,        'ANTONIO        DE        CAMARGO',
TO_DATE('25/01/1985','DD/MM/YYYY'),
    TO_DATE('12/08/2016','DD/MM/YYYY'),'RUA V, 44', 2451.33);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12359,        5,        'JOSE        DE        CAMARGO',
TO_DATE('23/10/1998','DD/MM/YYYY'),
    TO_DATE('20/04/2017','DD/MM/YYYY'),'RUA V, 88', 6541.22);

-- POPULAR A TABELA DEPENDENTE. ESCOLHER 3 FUNCIONARIOS E
CADASTRAR 3 DEPENDENTES
-- PARA CADA FUNCIONARIO ESCOLHIDO
-- FUNCIONARIO = 12345
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 1, 'JOANINHA', TO_DATE('12/10/2010','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 2, 'JULINHA', TO_DATE('15/10/2012','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
(12345, 3, 'TONINHO', TO_DATE('22/10/2014','DD/MM/YYYY'));

-- FUNCIONARIO = 12356
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 1,'JUNINHO', TO_DATE('02/10/2011','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 2, 'ZEZINHO', TO_DATE('15/10/2013','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 3, 'MARCELINHO', TO_DATE('17/10/2015','DD/MM/YYYY'));

-- FUNCIONARIO = 12359
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 1,'MARIAZINHA', TO_DATE('11/06/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 2,'LUIZINHA', TO_DATE('21/06/2016','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 3,'CARMINHA', TO_DATE('18/06/2018','DD/MM/YYYY'));

-- POPULAR A TABELA PROJETO COM 5 PROJETOS
-- 3 PROJETOS SEM A DATA TÉRMINO
INSERT INTO T_SIP_PROJETO VALUES
(10001,   'PROJETO   1',   TO_DATE('11/10/2016','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10002,   'PROJETO   2',   TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
INSERT INTO T_SIP_PROJETO VALUES
(10003,   'PROJETO   3',   TO_DATE('28/04/2012','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10004,   'PROJETO   4',   TO_DATE('25/03/2017','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
(10005,   'PROJETO   5',   TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- POPULAR A TABELA IMPLANTACAO
-- PROJETO 1
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    1,    12345,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('15/09/2017','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001, 2, 12346, TO_DATE('11/10/2016','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    3,    12348,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('25/06/2018','DD/MM/YYYY'));
-- PROJETO 2
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    1,    12345,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    2,    12352,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('25/07/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    3,    12355,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
-- PROJETO 3
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 1, 12354, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 2, 12355, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 3, 12345, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
-- PROJETO 4
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004, 1, 12357, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
```

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
(10004, 2, 12358, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004,    3,    12354,    TO_DATE('25/03/2017','DD/MM/YYYY'),
TO_DATE('15/08/2018','DD/MM/YYYY'));
-- PROJETO 5
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    1,    12345,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    2,    12350,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    3,    12354,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- CONSULTAR TABELAS
SELECT * FROM T_SIP_DEPARTAMENTO;
SELECT * FROM T_SIP_FUNCIONARIO;
SELECT * FROM T_SIP_DEPENDENTE;
SELECT * FROM T_SIP_PROJETO;
SELECT * FROM T_SIP_IMPLANTACAO;

--EFETIVAR AS TRANSAÇÕES PENDENTES
COMMIT;
```

Código-fonte 40 – Exemplo comando SQL – Populando as tabelas do banco de dados SIP
Fonte: Elaborado pela autora (2023)

1.17.1.2 Manipulando os dados com comandos DML

Após popular as tabelas na seção anterior, iremos exemplificar outras aplicações de manipulação de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_MANIPULACAO_PROJETO_SIP.SQL”, com a extensão SQL.

Sugerimos que o arquivo “SCRIPT”, seja elaborado na seguinte forma:

- As tabelas que possuírem colunas (campos) do tipo de dado “DATE”, deverão ter seus conteúdos inseridos/alterados/consultados, utilizando a função “`TO_DATE(<data>,<formato>)`”.
- Executar o SCRIPT e garantir que todos os comandos foram executados com sucesso.
- Após a execução dos comandos para popular as tabelas do banco de dados do projeto SIP, efetive as transações pendentes, criando e executando o comando “COMMIT”.

<u>Comando INSERT</u>: Escreva e execute a instrução abaixo e responda:

A inserção foi realizada com sucesso?

Qual foi a mensagem de erro exibida?

Justifique os motivos pelos quais a inserção não foi realizada com sucesso.

```
-- EXECUTE A INSTRUÇÃO
INSERT INTO T_SIP_DEPARTAMENTO VALUES(1,'FINANCEIRO','FIN');
```

Código-fonte 41 – Exemplo comando SQL – Treinando inserções
Fonte: Elaborado pela autora (2023)

Após a análise da execução, podemos responder:

A inserção não foi realizada.

Foi apresentada a mensagem de erro “`ORA-00001: restrição exclusiva (PF0102.PK_T_SIP_DEPARTAMENTO) violada`”.

A inserção não foi realizada com sucesso devido à existência de um registro (linha) com a chave primária informada na tabela “DEPARTAMENTO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Comando INSERT: Escreva e execute a instrução SQL, para inserir um novo registro (linha) na tabela “FUNCIONARIO”. Nesta inserção, insira um salário mensal inválido, por exemplo “VL_SALARIO_MENSAL=900” e responda:

A inserção foi realizada com sucesso?

Qual foi a mensagem de erro exibida?

Justifique os motivos pelo qual a inserção não foi realizada com sucesso.

```
-- EXECUTE A INSTRUÇÃO
INSERT INTO T_SIP_FUNCIONARIO
     VALUES (12380, 4,
             'LAURA DOS SANTOS',
             TO_DATE('10/03/1998','DD/MM/YYYY'),
             TO_DATE('24/10/2018','DD/MM/YYYY'),
             'RUA Y, 55',
             900);
```

Código-fonte 42 – Exemplo comando SQL – Treinando inserções
Fonte: Elaborado pela autora (2023)

Após a análise da execução, podemos responder:

A inserção não foi realizada.

Foi apresentada a mensagem de erro “`ORA-02290: restrição de verificação (PF0102.CK_SIP_FUNC_SALARIO) violada`”.

A inserção não foi realizada com sucesso devido à violação da “constraint check”, que avalia se o salário informado tem um valor maior ou igual a “1212,00”. Sendo assim, qualquer valor informado abaixo de “1212,00”, será recusado durante a inserção ou atualização dos dados.

Comando INSERT: Escreva e execute a instrução SQL, para inserir um novo registro (linha) na tabela “PROJETO”. Nesta inserção o projeto ainda não foi finalizado, portanto a coluna (campo) “DT_TERMINO”, deve ser preenchido com NULL de forma explicita.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- PREENCHIMENTO COM NULL DE FORMA EXPLICITA
INSERT INTO T_SIP_PROJETO
     VALUES (10006,
             'PROJETO 6',
             TO_DATE('05/04/2020','DD/MM/YYYY'),
             NULL);
```

Código-fonte 43 – Exemplo comando SQL – Treinando inserções
Fonte: Elaborado pela autora (2023)

Comando UPDATE: Escreva e execute a instrução SQL, para alterar os salários de todos os funcionários do departamento de código igual a 2 (dois), que após o dissídio coletivo, recebeu um aumento de 6,8%.

```
-- ATUALIZANDO OS DADOS DOS FUNCIONÁRIOS APÓS DISSÍDIO
UPDATE T_SIP_FUNCIONARIO
   SET VL_SALARIO_MENSAL = VL_SALARIO_MENSAL * 1.068
 WHERE CD_DEPTO = 2;
```

Código-fonte 44 – Exemplo comando SQL – Treinando atualizações
Fonte: Elaborado pela autora (2023)

Comando UPDATE: Escreva e execute a instrução SQL, para alterar os a data término de um projeto, devido a finalização das atividades e entrega do projeto ao cliente. O projeto que deverá ser finalizado, possui código igual a “10001” e foi finalizado na data "19/05/2021".

```
-- ATUALIZANDO O PROJETO FINALIZADO E ENTREGUE AO CLIENTE
UPDATE T_SIP_PROJETO
   SET DT_TERMINO = TO_DATE('19/05/2021','DD/MM/YYYY')
 WHERE CD_PROJETO = 10001;
```

Código-fonte 45 – Exemplo comando SQL – Treinando atualizações
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Comando DELETE: Escreva e execute a instrução SQL, para apagar registros (linhas) na tabela “IMPLANTACAO”. Esta deleção se faz necessária, devido a um cadastro incorreto de uma implantação, que foi associada a um funcionário erroneamente. A implantação que deve ser apagada tem o código de implantação igual a 2 (dois) e o funcionário em questão, tem o número de matrícula igual a “12358”.

```
-- DELETANDO IMPLANTAÇÃO
DELETE FROM T_SIP_IMPLANTACAO
 WHERE CD_IMPLANTACAO = 2 AND NR_MATRICULA = 12358;
```

Código-fonte 46 – Exemplo comando SQL – Treinando exclusões
Fonte: Elaborado pela autora (2023)

Comando ROLLBACK: Escreva e execute a instrução SQL, para descartar a deleção realizada da implantação, uma vez que o “COMMIT” ainda não foi executado. O funcionário que foi associado erroneamente não foi o de número de matrícula igual a “12358”, portanto teremos que descartar a alteração anterior.

```
-- DESCARTANDO A DELEÇÃO NÃO EFETIVADA
ROLLBACK;
```

Código-fonte 47 – Exemplo comando SQL – Treinando ROLLBACK
Fonte: Elaborado pela autora (2023)

Comando DELETE: Escreva e execute a instrução SQL, para apagar registros (linhas) na tabela “DEPARTAMENTO”. O departamento que deve ser apagado, tem o código igual a 2 (dois) e responda.

A exclusão foi realizada com sucesso?

Qual foi a mensagem de erro exibida?

Justifique os motivos pelo qual a exclusão não foi realizada com sucesso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- DELETANDO DEPARTAMENTO
DELETE FROM T_SIP_DEPARTAMENTO WHERE CD_DEPTO = 2;
```

Código-fonte 48 – Exemplo comando SQL – Treinando exclusões
Fonte: Elaborado pela autora (2023)

A exclusão não foi realizada.

Foi apresentada a mensagem de erro "`ORA-02292: restrição de integridade (PF0102.FK_SIP_FUNC_DEPTO) violada - registro filho localizado`".

A exclusão não foi realizada com sucesso devido à violação de restrição de integridade, ou seja, devido ao relacionamento entre a tabela "DEPARTAMENTO" e a tabela "FUNCIONARIO". Como temos funcionários associados ao departamento de código igual a dois, a deleção não é permitida.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S.B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C.A. **Projeto de Banco de Dados. Série Livros Didáticos.** v. 4. 6.ed. Rio Grande do Sul: Bookman, 2006.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados**: Implementação em SQL, PL/SQL e Oracle 11g. São Paulo: Pearson Education do Brasil, 2013.

PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2006.

SILBERSCHATZ, A; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

INSERT	Comando da linguagem SQL, responsável por inserir registros ou linhas nas tabelas.
UPDATE	Comando da linguagem SQL, responsável por atualizar registros ou linhas nas tabelas.
DELETE	Comando da linguagem SQL, responsável por excluir registros ou linhas nas tabelas.
SELECT	Comando da linguagem SQL, responsável por permitir a consulta ou recuperação de registros ou linhas nas tabelas.
DML	Data Manipulation Language (Linguagem de manipulação de dados)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com